



Introduction to Custom Electronics for VEXU

Revision 1 | 6/20/2020

Contents

Background **3**

Motivation **3**

Definitions **3**

Smart Port Communication (RS-485) **4**

RS-485 4

4P4C 4

Half vs Full Duplex 5

Example Code 5

COBS 5

Microcontrollers **7**

Microcontroller Selection 7

Microcontroller Integration 7

Board Design **7**

Introduction 7

PCB Design Software 8

Layers of a PCB 9

Traces 9

Vias 10

Planes 10

Surface Mount (SMD/SMT) vs Through Hole (THT) 11

Board Power 11

RS-485 Interface 14

Conclusion **15**

Additional Resources **16**

Background

Readers should be familiar with the following topics before proceeding with this document.

- PROS 3.2.0+
- Microcontroller or Arduino Experience
- Electronic Circuits Basics, Breadboard Experience
- Basic to Intermediate Level Soldering Experience

Motivation

Custom Electronics is a unique part of VEXU/VAIC and having a document with information about making custom boards is a useful tool to new teams and experienced teams wanting to expand their capabilities.

Definitions

Circuit Design:

PCB - Printed Circuit Board

THT - Through Hole Technology

SMT - Surface Mount Technology

SMD - Surface Mount Device

IC - Integrated Circuit

Trace - A flat copper wire on a PCB

Via - A small plated hole to connect traces on different layers.

Pad - An exposed copper section on a layer.

Hole - A hole; this could be for purely mechanical purposes (such as mounting), or have exposed plating for soldering a THT device.

Copper Layer - A thin layer of copper laminated to the board

Soldermask Layer - Layer on top of the copper layer, Gives color to the board, and covers everything but holes and pads.

Silkscreen - Normally White text or symbols to label parts or functions for human assembly.

Schematic - A diagram showing the connections between component pads/pins.

Net - A group of pads/pins of various components which (specified in the schematic) must be connected together.

Baud Rate - In digital systems Baud Rate is Bits per Second

Smart Port Communication (RS-485)

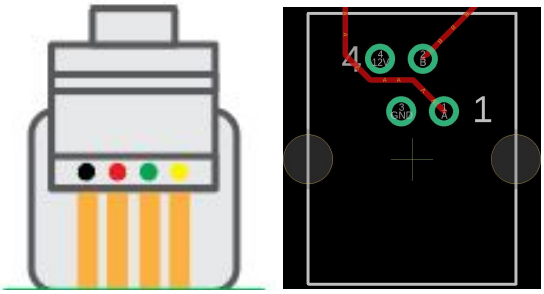
RS-485

RS-485 is a common electrical interface. The standard does not define a communication protocol. The advantages of RS-485 are that the differential voltage method allows for easier error correction and one master device can talk to multiple slave devices.

4P4C

4P4C is a type of Modular Connector and stands for 4 Position 4 Connector. There can be more positions than connectors but not the other way around. These are the pins and wire colors for the smart cables.

1	2	3	4
RS-485 A	RS-485 B	Ground	12V
Black	Red	Green	Yellow



Pictured Above (Left): 4P4C connector with color labels (Right): PCB footprint of a 4P4C connector

Half vs Full Duplex

Half Duplex and Full Duplex allow for back and forth communication but the difference is that full duplex allows for simultaneous back and forth. Example telephone(Full Duplex) vs walkie-talkie(Half Duplex).

Example Code

Example Code for the V5 and Teensy is in this github repo.
https://github.com/BCUZRobotics/Custom-Electronics-Example

COBS

Physical communication links aren't perfect. Every once in a while, a byte will be dropped by the UART-over-RS-485 link; when this happens, you want to be sure that your system can recover without requiring a hard reset. The key here is to ensure that your communication over the link happens in discrete "messages"; then, while it may not be possible to recover the contents of a message in which a byte was dropped in transit, you can at least ensure that subsequent messages are still able to be received and handled properly. One such scheme for implementing these requirements is called Consistent-Overhead Byte Stuffing (COBS).

COBS consists of an encoding algorithm for converting a message into the buffer to send over the link, and a decoding algorithm which does the opposite. Each algorithm takes a sequence of bytes as input and produces a sequence of bytes as output. The encoding process is as follows:

1. Insert a zero byte at the beginning of the sequence.
2. Insert a zero byte after each contiguous sequence of 254 non-zero bytes (even if that ends up with duplicate contiguous zero-bytes)
    a. Exception: don't add a zero-byte to the end in this step.
3. Insert a zero byte at the end of the sequence.

4. Replace every zero byte (except for the last one, which you added in step 3) with the offset to the next zero byte (i.e. the difference between the indices of the bytes within the sequence).
    a. Note: each offset is guaranteed to be at most 255—and therefore fit inside a byte—because of the transformation done in step 2

This will result in a sequence which only contains one zero byte, which is always at the very end, meaning that it is easy to split out messages coming in over a link. Also note the low number of bytes added: two, plus at most one per 254 bytes in the input. This means that the encoding is fairly efficient in use of bandwidth over the link.

Now, to decode a message, we must first ensure that we have a sequence of bytes which is in the form that we expect to come out of the encoding algorithm; that is, it has exactly one zero byte, which is at the end. In practicality, this would be done by accumulating bytes from the serial controller, and passing the entire buffer that we have accumulated into the decoding algorithm as soon as we put a zero byte, then clearing the buffer. The decoding process is as follows:

1. Remove the zero-byte from the end.
2. Create an index variable that starts at 0.
3. Get the value of the byte at the index, and overwrite that byte with zero.
4. Add the initial value of that byte onto the index.
5. If the index is within the bounds of the sequence (i.e. strictly less than the length), go back to step 3.
6. If the index is strictly greater than the length of the sequence, the encoding is malformed; abort.
7. Remove any zero-byte which follows 254 non-zero bytes.
8. Remove the zero-byte from the beginning of the sequence.

As you can see by comparing to the encoding process above, this directly mirrors its steps. Considering what would happen if a message got corrupted in transit, we can see that, in the worst case, that one message would be corrupted after decoding; in most cases, a dropped byte would cause the message to be discarded because an

index offset would not line up. However, it would not affect subsequent messages, meaning that you are able to recover from byte drops—or even complete temporary disconnects—cleanly, regardless of the length or internal structure of your messages. You can read more on COBS on its Wikipedia page.

Microcontrollers

Microcontroller Selection

Selecting a microcontroller for your application can be a very simple, or complex task. It depends on your requirements for your exact application. Most of this decision will be based on what communication technologies it has. To communicate with RS485 it needs at least one UART bus, so this is a requirement for all microcontrollers for use in custom electronics.

Microcontroller Integration

The two methods of microcontroller integration are mounting it as a component and building it into the board. The advantage to mounting it is that you can swap it if something goes wrong. The advantage to building into the board is that you can make the board smaller. A lot of microcontrollers have public schematics to build the microcontroller into the board with the right passive components. Arduino is a good example with public schematics and a public bootloader.

Board Design

Introduction

At first, PCB design will seem very daunting, even if you have experience designing electronic circuits on breadboards. Datasheets are your friend. For most applications, you can use the typical application circuit example. When jumping into advanced projects, like full integration of a microcontroller on your board, it may take a couple of tries.

For the purposes of this introduction document, we will be assuming you have a basic understanding of electronics, Arduino microcontrollers and breadboarding. If you do not have this, check the *Additional Resources* section at the end. There you will find some links to some educational material and videos about the topic.

Try to think of your PCB as a double-sided breadboard with flat wires. It doesn't have to contain support circuitry for a microcontroller or small surface mount components. It is up to you to decide what level of complexity you want your PCB to have.

PCB Design Software

To design your PCBs, you will need to use computer-aided design software that can output files a PCB manufacturer can interpret and manufacture. There are a few very good options for free educational software to use. These programs include Autodesk Eagle, EasyEDA and KiCad. All of these programs are great options for beginners learning how to design PCBs, but we would recommend starting on EasyEDA, for its ease of use, large public parts library, and it's the ability to work in your browser.

When you create an account you can create a new project. With the new project you are presented with a blank sheet for your schematic. On the schematic, right click and select "Place component". In the search bar you can find almost everything you need to make your board. A lot of schematics are available from LCSC but there are also many user contributed schematics. Be careful with user contributed schematics because they are often missing the PCB Package. When you have selected your components connect them in the schematic with the wire tool. When your schematic is complete you can convert it to a PCB. There will be a suggested pcb size and all the components from your schematic. There are thin blue lines connecting the nets. These are called ratlines. When you place your components you can start connecting them. Traces can go on the top and bottom but can only connect on the side where the SMT device is or either side for THT devices. There are good design practices which are

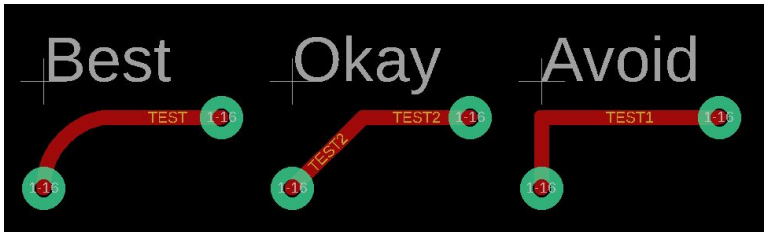
mentioned in later sections. A good video to watch is this one from Great Scott on YouTube.

Layers of a PCB

While it is possible to get PCBs with more than two layers, for most applications a 2 layer PCB will be the best option, as it is much cheaper and quicker to manufacture. On a 2 layer PCB, there are, well, more than 2 layers. There is a thin layer of copper that will be etched to act as the 'wires' between your components on each side of the PCB. This is then covered with a solder mask, which is similar to paint and is very electrically insulating. On top of the solder mask, there is an optional layer called a silkscreen. This is where your custom component names, values, or even logos can be added.

Traces

Traces are simply thin shapes etched out of a single piece of copper. These conductors are what carry electrical current between all components on your PCB. The amount of current-carrying capability of a trace is based on its width. This calculator will help you determine how wide your traces need to be. In general, you should allow the width of your traces between traces. So if your traces are 10 mils, you should allow 10 mils between your various traces. While, with modern manufacturing techniques it should never be a problem, you should avoid making traces that make 90 degree turns, as it could trap the acid used to etch the copper and cause a thinner than usual area.



Vias

A via is simply a plated hole that allows conduction from one layer of the PCB to the other. This allows you to switch the side a trace is on to avoid obstacles or to meet a component on the other side of the board. You should avoid placing vias directly on pads, as it is a good thermal conductor and will make soldering it harder.

Planes

A plane is a large surface dedicated to a single signal, such as ground or power, that is shared by multiple devices and circuits. It helps reduce ground loops and interference in components. Whenever possible these should be left continuous, and uninterrupted. If ground planes are used on both top and bottom layers, they should be ‘stitched’ together using vias at various places on the board. Planes are also very good at dissipating the heat components generate.

When generating planes, you will be given an option to use thermals, which means that applicable pads are connected to the surrounding plane by small spokes rather than solid copper, as seen in the layout below. Thermals are designed to reduce the thermal conductivity to the plane from the component, and make soldering much easier.

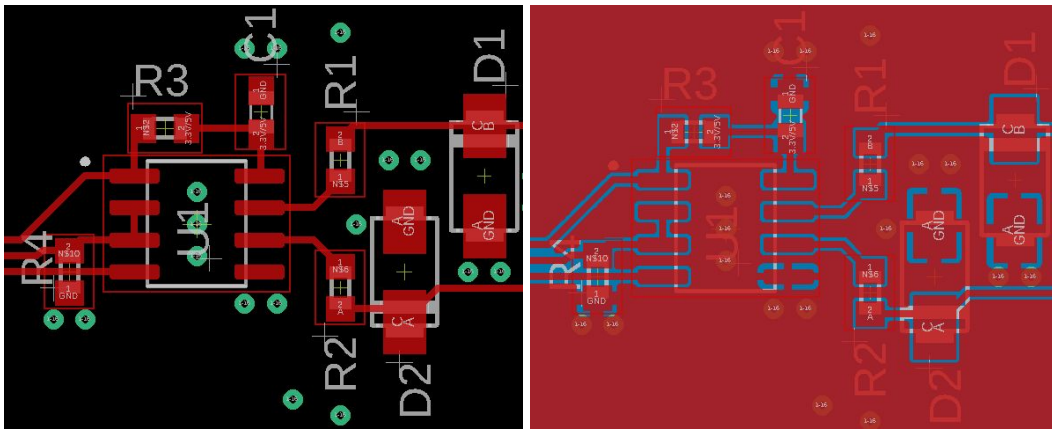


Figure A: Left: RS-485 Half Duplex Circuit without ground planes, Right: The same circuit using ground planes(with thermals) on both top and bottom layers.

Surface Mount (SMD/SMT) vs Through Hole (THT)

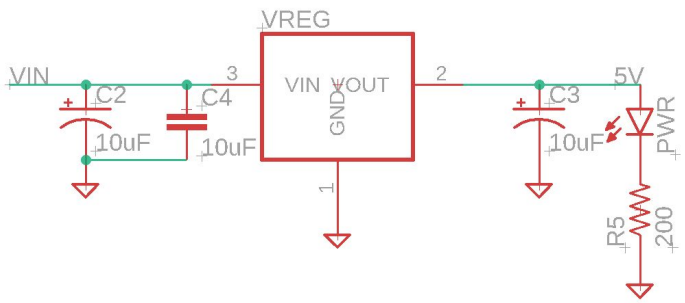
If you've worked with breadboards, chances are you've worked with Through Hole components. As their name entails, they go through a hole and are soldered to make electrical contact.

Surface mount components have several benefits, and in many cases, are the only available package type for some ICs. This is the case for almost all microcontrollers and IMUs. Surface mount components require intermediate soldering skills, as they are soldered to pads, instead of through holes. Some packages, like BGA and QFN, are very difficult to solder without the use of a hot air soldering station. In most cases, however, the packages are SOIC or SOT, both of which are reasonably easy for anyone with steady hands and a pair of tweezers.

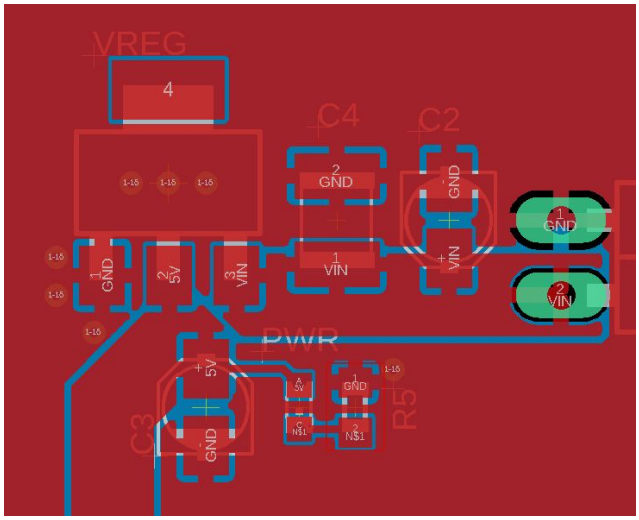
As to which package type you want to use, it is up to you. Using SMD will require more skill and will be more time consuming to assemble, but will open up to more advanced, and compact circuits.

Board Power

According to <VUR6c>, power can be sourced by either a lithium ion, lithium iron or nickel metal hydride battery pack of 12 volts max, or directly from a V5 Smart Port. There are pros and cons to both methods. A V5 Smart Port can output 12V @ 2.5 Amps which is plenty for most applications but could negatively affect power to motors, if the current draw is sufficient. An additional battery can supply higher current, and has the upside of not removing any potential power from the rest of your robot. For most simple microcontrollers, the maximum current that will be drawn is 75mA. Depending on which sensors you are using, your needs could be different. As a rule of thumb, assuming a total current draw of less than 250mA, a linear voltage regulator like this SOT-223 package, or this through-hole TO-220 package should work well for dropping the higher voltage down to logic level. You may need a different or additional regulator if you are using a 3.3V microcontroller.



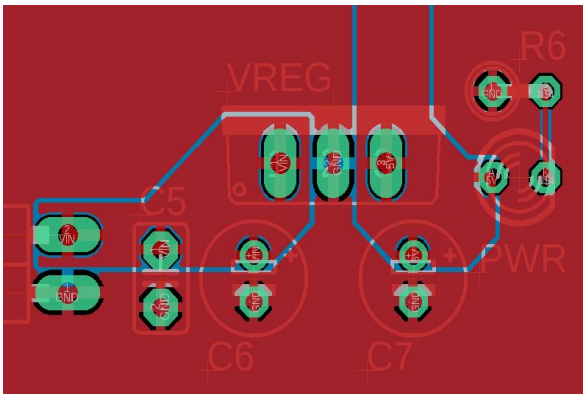
As per [datasheet recommendations], a capacitor should be placed at both the input and output side of the regulator, regardless of the package. A 10uF ceramic decoupling capacitor (This simply isolates the IC from the noise of other circuits) should also be added. Optionally shown on both the schematic and board layouts is an LED to indicate that the board has power, which is useful for basic debugging (and making sure you plugged it in before the match).



Above, Figure A: Voltage Regulator Schematic

Here is a basic layout example using SMD packages. Using SMD will reduce the footprint area, and allow the bottom layer to be used for a continuous ground or power plane, and the dissipation of heat.

Above, Figure B: SMD voltage regulator layout



Above, Figure C: THT voltage regulator layout

Here is an example of a board layout using through-hole components. It gives the additional benefit of being easier to solder and having the ability to carry more current, but it has the downside of taking up more space.

When choosing a linear voltage regulator, be it SMD or THT, it is important to not only look at the rated maximum current but also the power dissipation characteristics, specifically the thermal resistance. A linear voltage regulator works by taking all the excess power and dissipating it as heat, so this tells you how much the regular will heat up for each watt that it has to dissipate. For example, if you are going to be dropping 12V to 5V and drawing an average of 250mA, that means an average of (12V - 5V) * 0.250A = 1.75W needs to be dissipated by the regulator. If we look at the THT regulator that was mentioned earlier, we can see that it has a junction-ambient thermal resistance of 50 C/W for the TO-220 package version.

Symbol	Parameter	D²PAK	DPAK	TO-220	TO-220FP	Unit
R_{thJC}	Thermal resistance junction-case	3	8	5	5	°C/W
R_{thJA}	Thermal resistance junction-ambient	62.5	100	50	60	°C/W

This means that for every watt that it needs to dissipate, it will increase by 50C above ambient. So when dissipating our 1.75W, it will increase to 1.75W * 50 C/W = 87.5C above ambient. This is important because even though the regulator is rated at a maximum pique current of 1.5A, as you can see here, while drawing a continuous 250mA, it will heat to ~110C, which is about as hot as I'd want to run it. To help with

this, you could either add a heatsink and / or fan to help dissipate more heat, or better yet, switch to a much more efficient regulator such as a buck regulator.

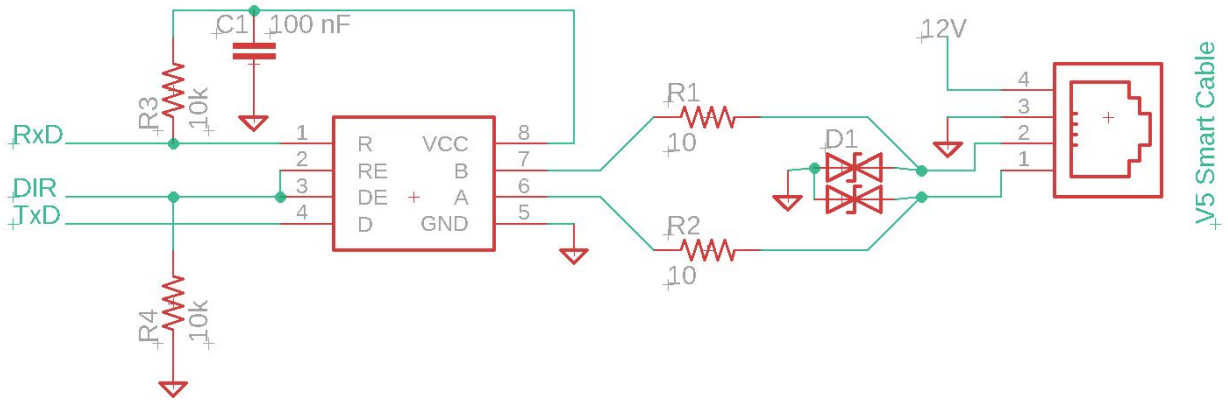
RS-485 Interface

The SN65HVD1782 RS-485 interface chip is the chip used by VEX in their smart port interfaces. For all intents in this document, we will be using the same chip. The datasheet for this chip can be found here.

All typical application schematics come from Figure 17 of the datasheet. A known problem with the V5 and Smart Ports is damage caused by static. This problem can be mitigated with additional external transient protection. A TVS (Transient Voltage Suppression) Diode, used in conjunction with a 10Ω resistor, will properly shield both the RS-485 transceiver and all electronics connected to it. Both CDSOT23-SM712 and SMBJ43CA are recommended by TI, however, any 60V+ 600W TVS Diode should work for this purpose.

As with other ICs, a decoupling capacitor should be used. A 100-220 nF bypass capacitor should be placed as close as possible to the V_{cc} of both the transceiver and any controllers connected to the circuit.

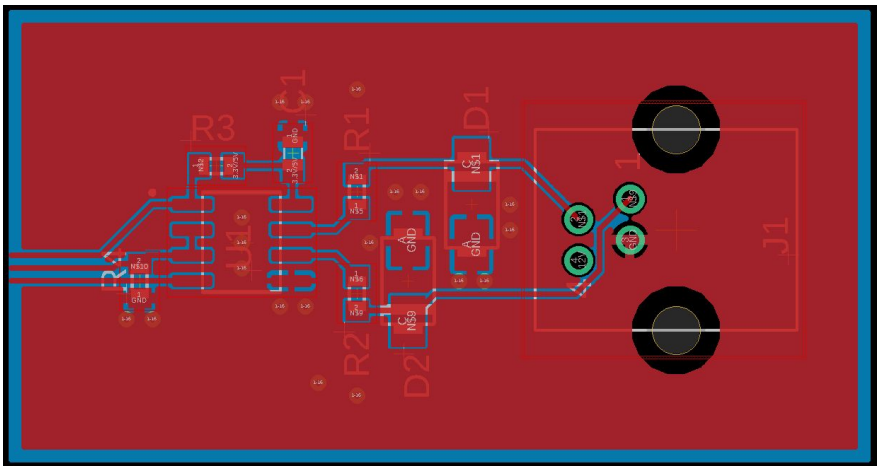
Resistors 3 and 4 are the pullup and pulldown resistors for the data lines. If interference in your RS-485 communication occurs, lowering the value of these resistors should help eliminate noise caused by transient voltages.



Above, Figure A: Half-Duplex RS-485 Transceiver Schematic w/ Transient Suppression

When laying out the PCB, attention should be given to the transient voltage suppression circuitry's placement. It should be located as close as possible to the connector. If your connector is on a different location on the board than your transceiver, the TVS diodes and resistors should be placed next to the connector.

To reduce via inductance, two vias should be used on connections from decoupling capacitors and TVS diodes to ground or power planes.



Above, Figure B: Half-Duplex RS-485 Transceiver w/ TVS & 4P4C Connector Board Layout

Conclusion

We hope you find this document useful and informative. If you have questions or suggestions feel free to contact us and we will be happy to help. We are on the Unofficial VEX Server and the VEX Advanced Wisdom Server. Our discord tags are majorjoel2#3434, Tixo Rebel#9147, Magik#6584, and MotorControllerBroke#4953. Best of luck to all teams.

Additional Resources

https://github.com/BCUZRobotics/Custom-Electronics-Example

https://www.digikey.com/product-detail/en/texas-instruments/SN65HVD1782DR/296-23802-1-ND/1957757

https://www.4pcb.com/trace-width-calculator.html

https://www.youtube.com/watch?v=35YuILUlfGs

https://en.wikipedia.org/wiki/Consistent_Overhead_Byte_Stuffing

https://jlcpcb.com/

https://www.youtube.com/watch?v=fq6U5Y14oM4